

Category B Problems — Reinforced with Basic Constructs

M1. Library Book Cataloguing

Scenario

A librarian receives a batch of book entries in one sitting — some already have a confirmed ISBN; some don't yet. Before this becomes an OOP problem at all, notice what it really is: for each entry in a list, check one condition (does it have an ISBN?) and act accordingly. That part, you already know how to do.

Requirement

- LibraryBook must offer two constructors linked through this() chaining — no duplicated field-setting logic between them.
- A book with no confirmed ISBN must default to "PENDING", never left blank or null.

Every entry in the batch must be processed and its status printed in a single pass.

Suggested Method Signature(s)

```
public LibraryBook(String title, String isbn)
public LibraryBook(String title) // chains via this(...)
```

Sample Input / Output

titles = {"Clean Code", "Untitled Draft", "1984", "Notes"} isbn = {"978-0132350884", "", "9780451524935", ""}	Clean Code 978-0132350884 Catalogued: true Untitled Draft PENDING Catalogued: true 1984 9780451524935 Catalogued: true Notes PENDING Catalogued: true

M2. Payroll Batch Bonus Round

Scenario

Payroll is giving every employee on a small team the same festival bonus this month. Strip away the class design for a second: this is “loop through a list, add a fixed amount to each person's number.” The this keyword challenge here is only about making sure that addition lands correctly inside the class, once you've already got the loop right.

Requirement

- Employee's constructor and raiseSalary() must each resolve a genuine field/parameter naming clash using this.
- Every employee in the array must receive the identical bonus amount in a single pass.

Each employee's final salary must be printed after the raise is applied.

Suggested Method Signature(s)

```
void raiseSalary(double salary)
```

Sample Input / Output

--	--

Category C · Page 1 of 3

CodInClub

Powered by BridgeLabz

4 employees with starting salaries {40000, 55000, 62000, 48000}, raiseSalary(5000) applied to each	E-101 Final Salary: Rs 45000.0 E-102 Final Salary: Rs 60000.0 E-103 Final Salary: Rs 67000.0 E-104 Final Salary: Rs 53000.0
--	--

M3. Late Fees — Skip the On-Time Accounts

Scenario

The finance office wants late fees calculated for a whole batch of accounts in one run — but only for accounts that are genuinely late. An account with daysLate equal to 0 (or somehow negative) should be skipped entirely, not charged a fee of Rs 0. That decision — process this one, skip that one — is plain if-else, nothing new. What's new is that the fee formula itself must be locked down so it can never quietly change.

Requirement

- calculateLateFee() and printSummary() must both be final, locked against ever being overridden by a subclass.
- An account with daysLate ≤ 0 must be skipped entirely, not charged a fee of Rs 0.
- The whole batch must be processed in a single pass, printing a summary or a skip message for every account.

Suggested Method Signature(s)

```
final double calculateLateFee(int daysLate)
final void printSummary(int daysLate)
```

Sample Input / Output

regNos = {"RA001","RA002","RA003","RA004"} totalFees = {200000,150000,180000,220000} daysLate = {10, 0, -2, 5}	RA001 Total Fee: Rs 200000.0 Late Fee: Rs 20000.0 RA002 - On time, no late fee RA003 - On time, no late fee RA004 Total Fee: Rs 220000.0 Late Fee: Rs 11000.0

M4. One-Time College Setup, Many Students

Scenario

SRM's student portal wants the college name and academic year loaded exactly once — the same static block idea you've already met. What's new here is scale: instead of typing new SrmStudent(...) twice by hand, you'll create a whole batch of students from a plain list of names, the same way you've built arrays of objects before.

Requirement

- collegeName and academicYear must be set exactly once, through a static block — never repeated per

object.

- Creating multiple students in a loop must not cause the static block to run more than once. •

Every student created in the batch must print a short confirmation line.

Suggested Method Signature(s)

```
static { ... }
```

Category C · Page 2 of 3

CodInClub

Powered by BridgeLabz

Sample Input / Output

names = {"Ravi", "Meera", "Karthik", "Divya", "Anitha"}	College info loaded Student record created: Ravi Student record created: Meera Student record created: Karthik Student record created: Divya Student record created: Anitha

M5. Account Batch Payments

Scenario

The finance office's nightly batch run has to process a whole list of accounts in one pass — some plain FeeAccount, some HostelFeeAccount — and each type pays differently. This is exactly a for loop with an if-else inside it, the same shape you've used all through this set; the only new part is that the “if” condition is an instanceof check instead of a number comparison.

Requirement

- processPayment() must use instanceof to correctly dispatch between HostelFeeAccount and a plain FeeAccount.
 - The batch must track how many of each account type were processed, using simple counters. •
- Both counters must be printed once, after the full batch has been processed.

Suggested Method Signature(s)

```
void processPayment(FeeAccount account, double amount)
```

Sample Input / Output

accounts = {Hostel, Hostel, FeeAccount, FeeAccount} (in that order), amount = 60000 for each	Paid in two installments (hostel account) Paid in two installments (hostel account) Paid in one go (day-scholar account) Paid in one go (day-scholar account) Hostel accounts processed: 2 Day-scholar accounts processed: 2

